

第十一、十二章 软件维护与软件项目管理

本章速览

核心一句话：**软件维护**是软件生命周期中**持续时间最长、成本最高**（占生存期工作量 **70%+**、达开发成本 **4 倍**）的阶段，本质是“为了改正错误或满足新需求而修改软件”；**软件项目管理**则是在**有限资源、有限时间内**通过**范围、成本、进度、质量**四要素的平衡以交付满足客户要求的软件产品，方法论核心是 **PMBOK 五大过程组 × 九大知识域**。

核心概念（必背 12 条）

1. **软件维护定义**：软件交付使用后，为改正错误或满足新需求而修改软件的过程。
2. **维护占生存期工作量**：**70% 以上**；大型软件维护成本 ≈ 开发成本的 **4 倍**。
3. **四类维护**（必背）：改正性、适应性、完善性、预防性。
4. **完善性维护占比最大**：约 **50%**。
5. **维护副作用三類**：代码副作用、数据副作用、文档副作用。
6. **逆向工程**：从已有程序**抽取**数据结构、体系结构、设计信息（设计恢复）。
7. **再工程**：= 逆向工程 + 重构 + 正向工程，将现存系统**重新构造**为新形式。
8. **项目四制约**：范围、成本、进度、客户满意度（质量）。
9. **PMBOK 五过程组**：启动、计划、执行、监控、收尾。
10. **PMBOK 九知识域**：综合、范围、时间、成本、质量、人力、沟通、风险、采购。
11. **代码行/功能点度量**：LOC 与 $FP = \text{总计数} \times (0.65 + 0.01 \sum F_i)$ 。
12. **期望工期**： $T_i = (a_i + 4m_i + b_i)/6$ 。

核心问题（高频考法）

- 四类维护如何辨析（Windows→Linux、性能优化、增加报表、重构组件）？ **选择/简答**
- "救火式维护"对应哪一类？ **选择**
- "维护 = 改 Bug"对吗？ **判断**
- 完善性维护占比是否最大？ **判断**
- 维护周期是否包含多次软件开发活动？ **判断**
- 逆向工程 vs 再工程的区别？ **简答**
- 软件项目管理 = 软件工程吗？ **判断**
- PMBOK 五过程组 + 九知识域是哪些？ **填空/简答**
- 项目目标受哪四个因素制约？ **简答**
- 期望工期 T_i 公式？ **填空/计算**
- 功能点 FP 公式？ **填空/计算**

高频考点（PPT 主导）

- ★ **四类软件维护的定义 + 经典场景对照**
(2007/2008/2009/2010/2011/2012/2013/2021 真题，**必考**)
- ★ **完善性维护占比最大约 50%**（判断/选择）
- ★ **维护占生存期工作量 70%+、成本 4 倍**
- ★ **"救火式维护 = 改正性维护"**（2013 真题）
- ★ **维护 ≠ 改 Bug；维护周期含多次开发活动**（2012/2013 判断）
- ★ **维护的副作用三類**
- ★ **项目四制约：范围/成本/进度/客户满意度**
- ★ **PMBOK 五过程组、九知识域映射**
- ★ **软件项目管理 ≠ 软件工程**（2012 判断）
- ★ **5 过程组 ≠ "启动/执行/监控/结束"**（漏计划）（2013 判断）
- ★ **任务期望工期公式 $T_i = (a + 4m + b)/6$**

题目关键词导航

第十一章 软件维护

关键词 / 场景	对应考点
"Windows→Linux / SQLServer→Oracle / 升级 OS"	适应性维护
"打印出错 / 系统崩溃 / 鼻子塌陷"	改正性维护
"增加新功能 / 增加报表 / 性能优化 / 学散打"	完善性维护
"重构组件 / 九阳神功重构内力 / 提高可维护性"	预防性维护
"救火式 / 紧急抢修 / 故障响应"	改正性维护
"维护 = 改 Bug"	×（错）
"完善性维护占比最大"	√（对，约占 50%）
"维护周期含多次开发活动"	√（对，IEEE 1219）
"逆向工程 / 再工程"	设计恢复 vs 系统重构
"维护副作用"	代码、数据、文档

第十二章 软件项目管理

关键词 / 场景	对应考点
"软件项目管理 = 软件工程"	×（PM 是 SE 的子集）
"PM 包括启动/执行/监控/结束"	×（漏"计划"）
"CMM 是衡量项目管理水平的标准"	×（是组织过程能力）
"缺乏项目管理经验 = 软件危机原因之一"	√（对）
"项目目标受制约"	范围/成本/进度/客户满意度
"PMBOK 五过程组"	启动/计划/执行/监控/收尾
"九大知识域"	综合/范围/时间/成本/质量/人力/沟通/风险/采购
"功能点 FP"	$FP = \text{总计数} \times (0.65 + 0.01 \times \sum F_i)$
"期望工期"	$T = (a + 4m + b)/6$
"WBS"	工作分解结构（范围管理）
"增加人力总能赶上进度"	×（《人月神话》）

知识主线

第十一章 软件维护主线

① 为什么需要维护? (生命周期最后阶段, 70%+ 工作量, 4 倍开发成本) → ② 维护是什么? (交付后为改错或满足新需求而修改软件) → ③ 维护分几类? (改正/适应/完善/预防, 完善占比最大 ~50%) → ④ 影响维护工作量因素 (系统大小、语言、年龄、文档) → ⑤ 维护流程 (IEEE 1219: 申请→分类→分析→设计→实现→系统测试→验收测试→交付) → ⑥ 维护期软件开发三步 (理解程序→修改程序→重新验证) → ⑦ 修改副作用 (代码/数据/文档) → ⑧ 逆向工程与再工程 (设计恢复 vs 系统重构)。

第十二章 软件项目管理主线

① 项目 / 软件项目 (时限性、唯一性、有限资源) → ② 项目管理 (应用知识/技能/工具在规定时间内完成目标) → ③ 四制约 (范围、成本、进度、客户满意度) → ④ PMBOK 九知识域 (综合/范围/时间/成本/质量/人力/沟通/风险/采购) → ⑤ 五过程组 (启动/计划/执行/监控/收尾, 过程组交互迭代) → ⑥ 过程组 × 知识域映射表 (核心考点) → ⑦ 软件度量 (产品/过程/资源; 内部/外部; 直接/间接) → ⑧ 规模度量 (LOC、功能点 FP) → ⑨ 项目估算 (基于分解、基于经验、COCOMO) → ⑩ 进度安排 (甘特图、PERT/CPM, 期望工期公式) → ⑪ 组织结构 (按课题/按职能/矩阵)。

核心知识点详解 (PPT 主导)

11.1 软件维护的定义 ★★★

定义 (教材原话): 软件维护是在软件已经交付使用之后, 为了改正错误或满足用户新的需求而修改软件的过程, 即在软件运行/维护阶段对软件产品所进行的一切改动。

为何需要维护:

- 软件生命周期中维护阶段持续时间最长;
- 隐藏错误会在特定环境下暴露;
- 外部环境 (软硬件、数据环境) 会变化;
- 用户会提出新功能/性能要求;
- 需要主动提高可维护性以延长寿命。

★考点 考点: 维护是交付后阶段一切修改活动; 维护≠改 Bug, 改 Bug 只是其中一类 (改正性)。

11.2 软件维护的代价 ★★★

- 大型软件平均维护成本 ≈ 开发成本的 4 倍;
- 国外软件开发组织 60% 以上人力用于维护已有软件;
- 软件维护工作量占整个软件生存期工作量的 70% 以上 (图 11-2);
- 维护阶段初期改正性维护较多, 随后进入稳定期, 适应性 + 完善性逐步增加。

■ 图示 图 11-1/11-2 信息: 图 11-1 饼图——三类维护占比: 完善性 50%+、适应性约 25%、改正性约 20% (圆心可加预防性 ~5%)。图 11-2 柱状对比——“维护阶段”占生存期总工作量 70%+, “开发阶段”占 30% 以下。

■ 避坑 不要把“70%”和“4 倍”两个数据搞混: 70% 是工作量比例, 4 倍是成本倍数。

11.3 四类软件维护 ★★★★★ (极高频必考)

① 改正性维护 (Corrective)

软件交付后必然有隐藏错误带到运行阶段。为识别和纠正软件错误、改正性能缺陷而进行的诊断和改正过程。

触发: bug、崩溃、错误结果。关键词: “出错”、“修复”、“诊断”、“救火式”。

② 适应性维护 (Adaptive)

为使软件适应外部环境变化 (新的软硬件配置、数据环境——数据库、数据格式、I/O 方式、存储介质) 而修改软件。

触发: OS 升级、数据库迁移、硬件更换。关键词: “移植”、“迁移”、“适应环境”。

③ 完善性维护 (Perfective)

为满足用户新的功能与性能要求, 扩充功能、提高性能、改进效率、提高可维护性而进行的维护。占比最大, 约 50%。

触发: 增加新功能、性能优化、改善交互。关键词: “新增”、“优化”、“扩充”、“提升性能”。

④ 预防性维护 (Preventive)

为提高可维护性、可靠性, 采用先进软件工程方法对软件重新设计、编制和测试。定义为“把今天的方法学用于昨天的系统以满足明天的需要”。占比很小。

触发: 重构、再工程、代码优化。关键词: “重构”、“采用先进技术”、“主动”。

★考点 辨析口诀: “错→改、境→适、新→完、防→预”。其中“新功能/性能提升”对应完善性, 最容易与适应性混淆。

11.4 影响软件维护工作量的因素 ★★

1. 系统大小: 系统越大、功能越复杂, 理解和改造难度越大。
2. 程序设计语言: 语言功能越强, 模块化/结构化程度越高, 所需指令数越少, 可读性越好, 维护越容易。
3. 系统年龄: 老系统不断修改→结构混乱; 维护人员更替→理解困难; 文档与实现脱节。
4. 其他: 应用类型、数学模型、任务难度、开关与标记、IF 嵌套深度、索引或下标数等。
5. 文档完备程度 (教材补充): 无文档或文档与程序不一致, 维护难度大增。

11.5 软件维护中的典型问题

- 问题定位困难 (开发环境无法重现生产数据/操作);
- 阅读理解别人写的程序非常困难;
- 需维护的软件往往文档不足或不合格;
- 不能指望原开发人员维护 (人员流动性大);
- 设计时未考虑将来修改→可维护性差;
- 修改一部分易引发其他部分异常;
- 维护工作不具吸引力, 易使人员产生挫折感。

11.6 维护机构 (教材 11.2.1)

典型维护机构由 7 种角色组成 (图 11-3):

1. 系统用户: 提出维护申请;
2. 维护管理员: 受理、评价、分配、验收, 是用户与维护机构的接口;
3. 系统分析人员: 分析问题、判断是否须修改;
4. 修改负责人: 执行修改以修复缺陷;
5. 测试人员: 充分测试, 确保问题解决且无副作用;
6. 工程人员: 重新部署系统;
7. 配置管理人员: 控制修改范围, 对软件配置审计。

注: 一人可承担多职; 明确责任可减少混乱。

11.7 维护申请单 MR / 软件修改报告 SCR

维护申请 MR (Maintenance Request): 由用户填写, 是用户与维护机构的交互手段。改正性维护须说明输入数据、操作流程、错误现象; 适应/完善性须附修改说明书。

软件修改报告 SCR (Software Change Report): 由软件组织内部给出, 说明修改性质、优先级、工作量、预计状况等。

11.8 IEEE 1219 软件维护工作流程 ★★★（教材 11.2.3）

1993 年 IEEE 计算机学会软件工程标准分委会颁布的《软件维护标准》：

- 1. 维护申请：由用户/程序员/管理人员提出；
- 2. 分类与鉴别：确认维护类别、给 MR 编号、入库；
- 3. 分析：先可行性分析（影响、方案、费用），后详细分析（更改需求说明、鉴别更改要素、提出测试策略、制定实施计划）；
- 4. 设计：更改设计基线、更改测试计划、修订详细分析结果、核实维护需求；
- 5. 实现：编码、单元测试、集成、风险分析、测试准备审查、更新文档；置于配置管理 SCM 控制下；
- 6. 系统测试：测试程序间接口；回归测试确保不引入新错误；
- 7. 验收测试：客户、用户、第三方共同进行，建立软件新版本；
- 8. 交付：安装、培训、备份、通知用户。

★考点 ★ 2013 真题：维护周期含多次软件开发活动（√）。依据：上述流程包含“设计、实现（编码/单元测试/集成）、系统测试、验收测试”。

11.9 维护期的软件开发（教材 11.3）

对源程序的修改须经历三步：

① 分析和理解程序

- 理解程序的功能与目标；
- 掌握程序的结构信息（系统结构、控制结构、数据结构、I/O 结构）；
- 了解数据流、控制流；
- 理解操作要求。

② 修改程序

先设计修改计划（规格说明、维护资源、人员、办公资源），自顶向下分析模块、接口、数据库；隔离待修改部分；详细设计修改逻辑；向用户提供回避措施。

然后修改代码。要求：正确编写、保持风格、注明改动语句、不删除可疑语句、不共用临时变量、插入错误检测语句、记录修改细节。

③ 重新验证程序

- 静态确认（≥2 人 review）；
- 计算机确认：先测修改部分→隔离测未修改部分→集成测试（回归测试）；准备标准测试用例；邀请用户测试；
- 维护后验收：文档完备、测试用例记载、配置副本、维护工序责任。

12.1 项目与软件项目 ★★

项目（Project）：为提供某项独特的产品、服务或成果所进行的有时限的工作。特征：

- 需要由人来完成；
- 受到有限资源的限制；
- 需要计划、执行和控制。

软件项目：成果体现为软件产品的项目。特有特征：

- 软件产品无形；
- 没有标准的软件过程；
- 大型软件项目开发常常是“一次性的”。

12.2 项目管理与软件项目管理

项目管理：把各种知识、技能、手段和技术应用于项目活动，以期在有限资源和规定时间内完成项目的特定目标。

软件项目管理：在软件开发过程中通过对各个开发环节的管理，在有限资源和规定时间内完成客户要求的特定软件产品。

★考点 ★ 2012 判断：项目管理 ≠ 软件工程。SE 包含开发技术与项目管理两大主线；PM 是 SE 的子集。

11.10 维护的三种副作用 ★★

副作用定义：因修改软件而造成的错误或其他不希望发生的情况。

- 1. 修改代码的副作用：删除/修改子程序、标号、标识符；改变时序；改变存储大小；改变逻辑运算符；修改文件“打开/关闭”；改进执行效率；边界条件变化等引入的错误。
- 2. 修改数据的副作用：修改数据结构导致软件设计与数据结构不匹配。如重新定义常量/记录/文件格式、增减数组大小、修改全局数据、重新初始化控制标志/指针、重排 I/O 或子程序参数。可通过详细设计文档+交叉引用控制。
- 3. 文档的副作用：修改程序未同步修改文档→文档与程序不匹配→默认条件改变、新错误信息不正确。交互输入顺序/格式改动未记入文档会引起重大问题。软件交付前对整个软件配置进行评审可减少文档副作用。

■ 避坑 对于用户来说，软件事实上就是文档。文档与代码不一致即视为软件缺陷。

11.11 软件逆向工程与再工程 ★★★

① 逆向工程（Reverse Engineering）

- 术语来自硬件：通过考察产品样品，导出其设计与制造的规格说明；
- 软件逆向工程 = 分析程序，恢复设计的过程；
- 从已有程序中抽取数据结构、体系结构、程序设计信息；
- 过程（图 13-7）：非结构化源代码→分析语句→重构代码→分析模式→结构化代码→理解处理/模块/界面→提取抽象→分析程序→初始规格说明→理解数据→分析系统→求精简化→最终规格说明。

② 再工程（Reengineering）

- 是一类软件工程活动，是工程过程；
- 把逆向工程 + 重构 + 正向工程组合起来，将现存系统重新构造为新的形式；
- 基础是系统理解（运行系统、源代码、设计、分析、文档全面理解）；
- 结果：与原程序功能相同，但质量更高；
- 是一个循环模型，每个活动可被重复，过程可在任一活动后终止；活动可线性或交错发生。

对比维度	逆向工程	再工程
本质	分析、抽取、恢复设计	重新构造为新形式
组成	仅分析与抽取	逆向 + 重构 + 正向
结果	规格说明/设计文档	新的、质量更高的系统
是否改动系统	不改	重新实现

12.3 项目目标与四制约 ★★★

项目目标：在一定时间、预算内完成工作的范围，以使客户满意。实现项目目标受四个因素制约：

- 1. 项目范围：为使客户满意必须做的所有工作；
- 2. 项目成本：完成项目所需费用，必须在客户资金限额以内；
- 3. 项目进度：安排每项任务的起止时间及所需资源等，是为项目描绘的过程蓝图；
- 4. 客户满意度：完成的项目质量是否达到预期效果（即质量）。

■ 图示 四要素关系图：常用等边四边形/正四面体示意——四个顶点（范围/成本/进度/质量）相互制约；任一顶点变化都会影响其余三个。质量（客户满意度）常置于中心。

12.4 PMBOK 九大知识域 ★★★★★

美国项目管理学会（PMI，1969）提出**PMBOK**（Project Management Body of Knowledge），1996/2000/2004/2008 多次修订。九大知识域：

#	知识域	核心内容（关键词）
1	综合管理	项目章程、项目管理计划、指导执行、监控、整体变更控制、项目结束
2	范围管理	范围规划、范围定义、 WBS 、范围确认、范围控制
3	时间管理	活动定义、活动排序、资源估算、历时估算、进度计划制定、进度控制
4	成本管理	成本估算、成本预算、成本控制
5	质量管理	质量规划、质量保证、质量控制
6	人力资源管理	人力资源计划、团队组建、团队建设、团队管理
7	沟通管理	沟通规划、信息发布、绩效报告、干系人管理
8	风险管理	风险规划、识别、定性/定量分析、应对规划、风险监控
9	采购管理	采购规划、招标询价、卖方选择、合同管理、合同收尾

■ **记忆** 口诀：**综范时成质，人沟风采**（综合·范围·时间·成本·质量·人力·沟通·风险·采购）。

12.5 PMBOK 五大过程组 ★★★★★

过程组	主要目标	典型活动
① 启动 Initiating	定义并批准项目或阶段；确定目标/范围	制订项目章程、制订项目范围说明书
② 计划 Planning	定义和细化目标，选择最优方案	项目管理计划、范围定义、WBS、进度计划、成本预算、风险计划、采购/合同计划等（ 内容最多 ）
③ 执行 Executing	整合人力及其他资源，完成项目活动	指导和管理执行、质量保证、团队建设、信息发布、获取供方响应、选择供方
④ 监控 Monitoring & Controlling	定期测量并监视，发现偏差并纠正	监督控制工作、整体变更控制、范围/进度/成本控制、质量控制、绩效报告、风险监控
⑤ 收尾 Closing	正式验收产品/服务/成果，有条件结束	项目收尾、合同收尾

★**考点** ★ 2013 判断：PM 含启动、**计划**、执行、监控、收尾——题干只列"启动/执行/监控/结束"**漏"计划"**→ ×（错）。

■ **记忆** 记忆：**"启计执监收"**。计划过程组是内容最多的过程组；各过程组非独立，通过输出/输入相互迭代关联。

12.6 过程组 × 知识域映射（核心表）★★★

知识域 \ 过程组	启动	计划	执行	监控	收尾
综合	章程、初步范围说明书	项目管理计划	指导管理执行	监控工作、整体变更控制	项目收尾
范围	—	范围规划、范围定义、 WBS	—	范围验证、范围控制	—
时间	—	活动定义/排序/资源估算/历时估算/进度制定	—	进度控制	—
成本	—	成本估算、成本预算	—	成本控制	—
质量	—	质量规划	质量保证	质量控制	—
人力	—	人力资源计划、团队组建	团队建设	团队管理	—
沟通	—	沟通计划	信息发布	绩效报告、干系人管理	—
风险	—	风险计划、识别、定性/定量分析、应对规划	—	风险监控	—
采购	—	采购规划、计划签约	请求供方回应、供方选择	合同管理	合同收尾

规律：**计划过程组**覆盖最广；监控主要为各域的"控制"；收尾只有综合（项目收尾）和采购（合同收尾）有活动。

12.7 软件项目度量（PPT）★★

度量：根据已明确规则把数字或符号指定给实体属性，以阐述实体状态。

度量实体三大类：

- **产品**：规格说明、设计模型、代码、测试用例等中间产品；
- **过程**：编制规格说明、详细设计、测试等活动；
- **资源**：人员、团队、软硬件、办公地点。

属性维度：

- **内部属性**：能纯粹用实体自身度量（如规模、可复用性、耦合度、内聚度）；
- **外部属性**：由实体与环境共同度量（如质量、复杂性、可维护性）。

度量方式：

- **直接度量**：不依赖其他属性（如代码行数）；
- **间接度量**：与其他属性有关（如功能点、生产率）。

12.8 软件规模度量 ★★★

① 代码行 LOC 度量

代码行 = 所有可执行的源代码行数，包括可交付的动作和控制语句、数据定义、数据类型声明。

派生指标：

- 生产率 = KLOC / 工作量（人月）；
- 质量 = 错误数 / KLOC；
- 单位成本 = 成本 / KLOC；
- 单位文档 = 文档页数 / KLOC。

② 功能点 FP 度量

由 IBM 工程师 Allan Albrecht 在 1970s 提出，是生产率度量法，利用程序的功能性/实用性和数据域计数导出。

五个数据域特征计数：

1. 外部输入数 EI：用户输入；
2. 外部输出数 EO：用户输出；
3. 外部查询数 EQ：联机输入；
4. 内部逻辑文件数 ILF：逻辑主文件；
5. 外部接口文件数 EIF：传送到另一系统的接口。

调整后功能点公式：

$$FP = \text{总计数} \times \left(0.65 + 0.01 \times \sum_{i=1}^{14} F_i \right)$$

其中 F_i 为 14 个环境因素/复杂性调节因素（0~5 取值）。

★考点 ★ 考点：FP 公式系数 $0.65 + 0.01 \cdot \Sigma F_i$ ；FP 是间接度量，与编程语言无关。

12.9 软件项目估算 ★★★

估算四步：

1. 明确项目范围：功能、性能、约束、接口、可靠性；
2. 估算项目资源：人力、开发环境、可复用构件；
3. 估算成本和工作量：基于规模和历史经验建立基线；
 - 基于分解技术的估算模型；
 - 基于经验的估算模型；
 - COCOMO 模型（Constructive Cost Model）。
4. 确定开发时间：基于甘特图和 PERT 技术确定关键路径，得最短/最合理/最长开发时间。

COCOMO 模型（补充）：是一种基于代码行（KLOC）的精确成本估算模型，分为基本/中间/详细三个层次，将软件分为有机/半分离/嵌入三种类型，给出工作量（人月）= $a \cdot (KLOC)^b$ 与开发时间公式。

12.10 软件项目进度安排 ★★★

进度计划需用图示法描述，特别是任务间的依赖关系。需要表示：

- 各任务的计划开始/完成时间；
- 各任务完成的标志（○文档编写、△评审）；
- 参与人数、工作量衔接；
- 物理资源和数据资源。

常用工具：甘特图（Gantt）、PERT/CPM 网络图。

任务完成时间三估值

- 乐观时间 a_i ：顺利情况；
- 最可能时间 m_i ：正常情况；
- 悲观时间 b_i ：最不利情况。

期望工期公式：

$$T_i = \frac{a_i + 4m_i + b_i}{6}$$

★考点 ★ 必背： $T = (a + 4m + b) / 6$ 。系数 4 对最可能时间加权；标准差 $\sigma = (b - a) / 6$ 。

■ 避坑 2011 判断：进度延期时总可以通过增加人力赶上进度 → ×（错）。《人月神话》：人月不能简单换算，加人会增加沟通成本。

12.11 软件项目组织结构（PPT）★

组织原则三条：

1. 尽早落实责任：尽早指定专人负责，使他有权有责；
2. 减少接口：生产率与沟通途径数成反比；
3. 责权均衡：经理责任与权力对等，避免有责无权/有权无责。

三种组织模式：

1. 按课题划分：人员按课题组成小组，自始至终参加课题各项任务；
2. 按职能划分：人员按工作阶段分专业小组（设计组、编码组、测试组等），产品沿工序流水线传递；
3. 矩阵模式：按工作性质成立专门组（开发/业务/测试/维护），同时每个项目有经理；每个人既属专门组又参加项目。

■ 图示 矩阵模式图：行 = 专门组（开发、业务、测试、维护），列 = 子项目 1/2/3；交点为软件人员，归“专门组”和“项目”双重管理；上方为总经理，每个项目由产品经理负责。

补充知识点（教材独有）

补-1 软件工程方法学的目标

软件工程方法学的一项**主要目标**是**提高软件的可维护性**、减少软件维护所需的工作量、降低软件系统的总成本。

补-2 维护工作量随时间变化

维护阶段的最初一、二年，**改正性维护**工作量较大（隐藏 bug 集中暴露）；错误发现率降低、系统稳定后进入**正常使用期**；之后**适应性 + 完善性**逐步增加。

补-3 维护工作管理规程

包括：建立维护机构；确定维护申请报告的流程及评价过程；为维护申请规定标准的处理步骤；建立维护活动的等级制度及评审标准。维护工作没有严格时间规律，涉及人员多、周期长、流程复杂。

补-4 修改计划四要素

- 1. 规格说明信息（数据修改、处理修改、JCL 修改、接口修改）；
- 2. 维护资源（新程序版本、测试数据、所需软件系统、计算机时间）；
- 3. 人员（程序员、用户相关、技术支持、厂家联系、数据录入）；
- 4. 办公资源（纸面、计算机媒体）。

补-5 回避措施（修改期间）

- 原因未找到时：先就问题现象提供**回避的操作方法**，让用户业务可继续；
- 原因已找到时：通过**临时修改或改变运行控制**回避系统运行问题。

补-6 项目干系人

项目干系人（Stakeholder）：积极参与项目或其利益因项目实施而受影响的人员或组织。

关键干系人：

- 项目经理；顾客/客户；执行组织；
- 项目团队成员；项目管理团队；
- 出资人；有影响力的人。

积极干系人能从项目中获益；消极干系人因项目利益受损须**提前识别、提前沟通**。

核心对比表

对比-1 四类软件维护（★★★★★ 最高频）

类型	定义	起因	关键词	典型场景
改正性 Corrective	识别并纠正 隐藏错误 、改正性能缺陷	交付后暴露的 bug	"出错"、"诊断"、"修复"、"救火式"	打印 100 份出错、整容鼻子塌陷、系统崩溃
适应性 Adaptive	适应 外部环境/数据环境变化	软硬件/OS/数据库迁移	"移植"、"迁移"、"适应环境"	Windows→Linux、SQLServer→Oracle、水土不服
完善性 Perfective	扩充功能、 提高性能 、改进效率（占比~50%）	用户新需求	"新增"、"优化"、"扩充"、"提升"	增加报表功能、性能优化、加移动办公、学散打
预防性 Preventive	采用先进方法 主动重构 ，提高可维护性/可靠性	未来易于维护	"重构"、"采用先进技术"、"主动"	重构报表组件、九阳神功重构内力

■ **记忆** ★ 占比排名：完善性（~50%） > 适应性（~25%） > 改正性（~20%） > 预防性（<5%）。

补-7 沟通的分类

- 口头 vs 书面；
- 正式 vs 非正式；
- 单向 vs 双向；
- 语言 vs 非语言。

经验：项目经理 **80% 以上**时间用于各种沟通活动。

补-8 软件开发的三方面平衡

软件项目管理最重要的三方面：**时间、成本、质量**。在规定时间内、预算成本内、开发出高质量软件是企业追求的目标。三者**相互影响、相互制约**，良好管理须综合平衡。

补-9 ISO 软件质量模型 6 方面

功能性、可靠性、易用性、效率、可维护性、可移植性。

补-10 风险定义与分类

风险：损失发生的不确定性，是对潜在的、未来可能发生的损害的度量。风险不一定会发生，但发生则造成损害。

项目风险管理的目标：**提高积极事件概率/影响，降低消极事件概率/影响**。

补-11 采购的两类

- 1. 市场流通软件产品采购（数据库、中间件等）；
- 2. 针对特殊需求的服务采购（外包测试、外包开发）。

补-12 CMM 与 PM 关系

软件能力成熟度模型（CMM/CMMI）是衡量**软件组织过程能力/过程成熟度**的框架，覆盖软件工程全生命周期的多个过程域。**PM 只是其中一部分**（如 CMM 2 级"可重复级"重点在项目管理关键过程域，3-5 级覆盖更广）。

■ **避坑** ★ 2007 判断："CMM 是衡量项目管理水平的标准" → ×（错，是**过程能力**而非仅 PM）。

对比-2 逆向工程 vs 再工程

维度	逆向工程	再工程
本质	分析程序、抽取设计	工程过程，重新构造
目的	恢复设计规格说明	构建新的、更高质量系统
组成	仅分析与抽取	逆向 + 重构 + 正向工程
是否修改源码	否（只读分析）	是（重新实现）
基础	源代码	对系统全面理解
输出	设计文档/规格说明	新版本系统
模型特点	单向线性流程	循环模型，可终止于任一活动

对比-3 维护副作用三类

副作用	触发	典型错误	控制手段
代码副作用	修改子程序/标号/标识符、改时序、改逻辑运算符、改边界	运行错误、逻辑错误	多人 review、回归测试
数据副作用	改数据结构、改常量、改记录/文件格式、改全局数据	设计与数据不匹配	详细设计文档、交叉引用表
文档副作用	修改程序未同步修改文档	文档与程序不一致、用户操作错误	对整个软件配置评审

对比-4 五大过程组

过程组	核心目标	主要活动数	典型输出
启动	定义并批准项目	2	项目章程、初步范围说明书
计划	细化目标、选最优方案	最多 (20+)	项目管理计划、WBS、进度/成本/质量/风险计划
执行	整合资源完成活动	8	可交付物、绩效数据
监控	测量监视、纠偏	12	变更请求、绩效报告
收尾	正式验收、结束项目	2	最终产品、归档文件

对比-5 度量方法分类

维度	分类	说明
按属性来源	内部属性	仅实体自身 (规模、耦合、内聚)
	外部属性	需结合环境 (质量、可维护性)
按度量方式	直接度量	不依赖其他属性 (LOC)
	间接度量	依赖其他属性 (FP、生产率)
规模度量	LOC 代码行	简单但忽略功能复杂度
	FP 功能点	与语言无关, 基于 5 数据域

对比-6 进度安排三种组织模式

模式	划分依据	优点	缺点
按课题	按项目/课题	责任清晰、全程参与	专业技能不集中
按职能	按工作阶段	专业熟练、流水线	交接多、责任分散
矩阵	课题 × 职能	资源灵活、专业共享	双重领导、冲突多

开放题答题模板

开放-1 场景判别维护类型 (5 分)

题型：给出多个事件, 要求判断各属哪一类维护。

答题模板 (每事件三步走)：

1. 定性：xx 维护；

2. 引用定义：xx 维护是"....." (套用教材定义)；

3. 对号入座：题中"....."对应 xx 维护的"触发条件", 故为 xx 维护。

四类口诀：错→改、境→适、新→完、防→预。

例：Windows→Linux = 适应性 (外部环境变化)；增加报表功能 = 完善性 (扩充功能)；重构组件 = 预防性 (采用先进技术提高可维护性)；打印出错 = 改正性 (隐藏 bug)。

开放-2 软件项目失败的主要原因

答题要点 (5 条)：

1. 缺乏软件项目管理经验——是软件危机主要原因之一 (2015 判断考点)；

2. 需求不清晰、频繁变更——范围失控；

3. 估算与进度安排不当——工期过紧、盲目"加人"导致 Brooks 定律失效；

4. 缺乏干系人沟通——PM 80% 时间用于沟通, 沟通不到位致偏离用户期望；

5. 忽视质量保证与风险管理——后期集中暴露缺陷；无风险识别与应对计划。

可延伸：解决方案应围绕 PMBOK 九知识域 + 五过程组展开, 强调计划过程组、范围控制、风险识别。

开放-3 维护期发现设计问题, 如何处理? (2021 真题)

答题三段：

1. 按 IEEE 1219 维护流程解决：维护申请 → 分类鉴别 → 可行性分析 + 详细分析 → 设计 (更改基线、更新测试计划) → 实现 (编码、单元测试、集成, 置于 SCM) → 系统测试 (含回归测试) → 验收测试 → 交付。

2. 部署前测试活动：静态确认 (多人 review) → 单元测试 → 集成测试 → 回归测试 (确保不引入新错误) → 确认测试 → 系统测试 → 验收测试。

3. 部署方案：全量发布、蓝绿部署、灰度/金丝雀发布、滚动部署、特性开关等, 须配套回滚机制 + 监控告警。

开放-4 什么是软件项目管理? 制定项目计划包含的过程?

定义：软件项目管理是在软件开发过程中通过对各个开发环节的管理, 在有限资源和规定时间内完成客户要求的特定软件产品。

制定项目计划包含的过程 (属计划过程组, 覆盖 9 知识域规划类活动)：

1. 综合：制订项目管理计划；

2. 范围：范围规划、范围定义、WBS；

3. 时间：活动定义/排序/资源估算/历时估算/进度计划制定；

4. 成本：成本估算、成本预算；

5. 质量：编制质量计划；

6. 人力：人力资源计划编制、团队组建；

7. 沟通：沟通计划编制；

8. 风险：风险计划、识别、定性/定量分析、应对规划；

9. 采购：采购规划、合同计划。

开放-5 维护副作用如何控制?

1. 代码副作用：按配置管理控制修改；多人 review；回归测试验证未引入新错误。

2. 数据副作用：建立交叉引用表, 把数据元素、记录、文件与其他结构联系起来；详细设计文档控制。

3. 文档副作用：交付前对整个软件配置进行评审；文档与代码同步修改。

开放-6 解释 PMBOK 五过程组与九知识域的关系

答：五过程组 (启动/计划/执行/监控/收尾) 描述项目时间流程, 九知识域描述项目专业领域。两者构成二维矩阵——每个具体管理活动都是某过程组与某知识域的交叉点。例如："制订 WBS"是计划过程组 × 范围管理的交叉；"成本控制"是监控过程组 × 成本管理的交叉。

规律：计划过程组活动最密集；监控过程组的活动多为各域的"控制"；启动和收尾活动最少。

开放-7 给出 FP 计算题

题型：给出 5 数据域计数和 14 个环境因子 Fi 之和, 求 FP。

模板：

1. 先求总计数 = Σ(各域计数 × 权重)；

2. 套公式：FP = 总计数 × (0.65 + 0.01 × ΣFi)；

3. 注意 ΣFi 取值范围 0~70 (14×5)。

特例：若 ΣFi=0, 则 FP = 0.65 × 总计数 (环境最简单)；若 ΣFi=70, 则 FP = 1.35 × 总计数 (环境最复杂)。

历年真题汇总与详解

【2007–2008 期末 单选】某企业的软件系统希望从 Windows 平台移植到 Linux 平台上，软件开发厂商为了满足企业的要求进行的维护属（ ）

A. 改正性维护 B. 适应性维护 C. 完善性维护 D. 预防性维护

标签：B（适应性维护）。

解析：教材 11.1.1 节明确给出适应性维护定义——“为了使软件适应外部环境（新的软硬件配置）或数据环境（数据库、数据格式、I/O 方式、存储介质）的变化而修改软件的过程称为适应性维护”。Windows→Linux 属于典型的“操作系统/外部环境变化”，符合适应性维护范畴。改正性是改 bug；完善性是新增功能或提升性能；预防性是主动重构。

【2007–2008 期末 判断】软件维护的周期远远大于软件开发的周期。（ ）

标签：√（对）。

解析：教材 11.1 节明确指出“软件维护工作占整个软件生存期工作量的 70% 以上”，且大型软件维护成本高达开发成本的 4 倍。维护阶段从软件交付使用开始，直到软件被废弃为止，持续时间远长于需求+设计+编码+测试的开发周期。

【2007–2008 期末 判断】软件能力成熟度模型是衡量软件项目管理水平的标准。（ ）

标签：×（错）。

解析：CMM/CMMI 是衡量软件组织过程能力/过程成熟度的框架，覆盖软件工程全生命周期的多个过程域（需求、设计、编码、测试、配置管理、过程管理、培训等）。项目管理只是其中的一个方面（如 CMM 2 级“可重复级”重点在 PM 关键过程域，3–5 级覆盖更广）。题干将 CMM 仅归为“衡量 PM 水平”过于狭窄。

【2007–2008 期末 简答】什么是软件项目管理？制定项目计划时包含的项目管理过程有哪些？

软件项目管理定义（教材 12.1）：在软件开发过程中通过对各个开发环节的管理，在有限资源和规定时间内完成客户要求的特定软件产品。

制定项目计划包含的过程（计划过程组 + 9 知识域的规划类活动）：

1. 综合：制订项目管理计划；
2. 范围：范围规划、范围定义、WBS；
3. 时间：活动定义、活动排序、活动资源估算、活动历时估算、制订进度计划；
4. 成本：成本估算、成本预算；
5. 质量：编制质量计划；
6. 人力：人力资源计划编制、组建项目团队；
7. 沟通：沟通计划编制；
8. 风险：编制风险管理计划、风险识别、定性风险分析、定量风险分析、制订风险应对计划；
9. 采购：制订采购计划、编制合同。

【2008–2009 期末 单选】某企业的软件系统投入运行一段时间后出现响应速度慢的情况，软件开发厂商为了适应企业的要求，对软件性能进行了优化，提高了运行速度，企业进行的维护工作属于（ ）

A. 适应性维护 B. 完善性维护 C. 改正性维护 D. 预防性维护

标签：B（完善性维护）。

解析：教材 11.1.1 节完善性维护定义——“为了满足这些要求，须修改或再开发软件，以扩充软件功能、提高软件性能，改进加工效率，提高软件的可维护性”，且完善性维护在所有维护中占比最大。本题“优化性能、提高运行速度”完全符合完善性维护特征。注意：题目“为了适应企业的要求”中的“适应”是日常用语而非“适应性维护”的术语含义；适应性维护特指适应外部环境变化。

【2009–2010 期末 单选】某企业的软件系统投入运行一段时间后出现响应速度慢的情况，软件开发厂商为了适应企业的要求，对软件性能进行了优化，提高了运行速度，企业进行的维护工作属于

A. 适应性维护 B. 完善性维护 C. 改正性维护 D. 预防性维护

标签：B（完善性维护）。

解析：与 2008–2009 题完全相同。“系统运行正常，只是速度慢，厂商主动优化性能以满足用户新要求”——既非改正错误，也非环境变化，而是为提升软件性能而做的工作，对应完善性维护。完善性维护占比最高（~50%）。

【2010–2011 期末 单选】中央空调系统初期完成后，需要增加计费 and 报表功能，软件开发厂商为了满足要求进行的维护属于

A. 改正性维护 B. 适应性维护 C. 完善性维护 D. 预防性维护

标签：C（完善性维护）。

解析：完善性维护是为满足用户提出的新功能/性能要求而修改或再开发软件，以扩充软件功能、提高性能。本题“增加计费和报表功能”属于典型的新功能扩充，对应完善性维护。改正性是修错误；适应性是适应环境变化；预防性是为提高可维护性主动重构。

【2011–2012 期末 简答】学校向某公司购买了一套办公管理系统，运行期间发生了如下事件：(1) 系统运行两月后，需要将 SQLServer 数据库更换为 Oracle 数据库；(2) 教务处发现一次性打印 100 份试卷会出错；(3) 财务处要求在系统中增加移动办公功能；(4) 软件公司采用更先进的技术将报表组件重构以增强灵活性。请说明以上四类事件分别对应于哪一类软件维护工作。(5 分)

标签：

1. 适应性维护——数据库环境发生变化（SQLServer→Oracle），为适应外部数据环境变化而修改软件；
2. 改正性维护——一次性打印 100 份试卷出错是交付后暴露的隐藏错误，需诊断并改正；
3. 完善性维护——用户新提出功能要求（增加移动办公），为扩充功能而进行；
4. 预防性维护——采用更先进技术重构报表组件，为提高可维护性/可靠性，为以后改进打下基础。

解析：教材 11.1.1 节给出 4 类维护的定义：改正性（纠错）、适应性（适应环境变化）、完善性（扩充功能/性能，占维护工作量最大约 50%）、预防性（采用先进方法学重新设计）。四个事件分别精确对应。评分要点：(1) 2 分；(2)(3)(4) 各 1 分。

【2011–2012 期末 判断】在项目面临进度延期的情况下，总是可以通过增加人力在后期跟上进度。

标签：×（错）。

解析：软件项目的人力与时间不能简单互换（Brooks 定律 / 《人月神话》）。增加人员会带来培训成本、沟通成本上升，新人在短期内反而可能拖慢进度。题目中“总是可以”过于绝对，是错误判断。

【2012–2013 期末 判断】软件维护活动不包括软件开发活动。

标签：×（错）。

解析：教材 11.3 节标题即为“维护期的软件开发”，明确指出“进行软件维护时，必然对源程序进行修改”，且修改需经历“分析和理解程序、修改程序、重新验证程序”三步；11.2.3 节给出的 IEEE 1219 维护工作流程中包含“设计、实现（编码、单元测试、集成）、系统测试、验收测试”等活动，与开发活动高度重合。完善性维护与预防性维护尤其涉及大量再开发工作。故“维护不包括开发”错误。

【2012-2013 期末 简答】留学韩国的小明新交一韩国女友如花，交往期间发生了如下事件：(1) 小明希望如花学会中华散打技能；(2) 如花习武期间，去年整容的鼻子不幸塌陷，需要重新修补；(3) 毕业后如花随小明回北京，由于水土不服，如花卧病，经多次调理后康复并习惯中国生活；(4) 如花学习更高深的九阳神功，重构内力以增强体质。假设把如花比作软件，请说明以上四类事件分别对应于哪一类软件维护工作。(5 分)

标签：

1. 完善性维护——新增"中华散打"是用户提出的新功能要求，**扩充原有能力**；
2. 改正性维护——整容的鼻子塌陷属于交付后才发现的**隐藏缺陷**，须诊断和纠正；
3. 适应性维护——"水土不服/换环境"对应**外部环境变化**，须修改软件使之适应新环境；
4. 预防性维护——"重构内力以增强体质"对应为提高可维护性、可靠性而进行的主动**再设计/再工程**。

解析：教材 11.1 节给出 4 类维护的定义。本题以生活化类比考查分类判定。评分要点：(1) 2 分；(2)(3)(4) 各 1 分。注意 (3) 中的"多次调理后康复"易与"改正性"混淆——核心是**环境变化**触发，故为适应性。

【2012-2013 期末 判断】软件的项目管理就是软件工程。

标签：× (错)。

解析：软件工程包含**软件开发技术**（方法学、过程模型、工具、环境）与**软件项目管理**两大主线，项目管理仅是软件工程的一个组成部分。教材 1.3.1 节在给出软件工程定义时强调"软件工程涉及软件产品的所有环节。人们往往偏重于软件开发技术，**忽视软件项目管理的重要作用**"——可见二者并列而非等同。

【2013-2014 期末 判断】软件维护周期中包括多次的软件开发活动。

标签：√ (对)。

解析：教材 11.2.3 节给出的 IEEE 1219 维护工作流程包含 8 个步骤：维护申请、分类与鉴别、分析、**设计、实现（编码、单元测试、集成、风险分析、测试准备审查、更新文档）**、系统测试、验收测试、交付。其中设计、实现、系统测试、验收测试都是典型的软件开发活动，每次维护申请都会触发这些活动，故维护周期中包含"多次的软件开发活动"。

【2013-2014 期末 单选】下面哪一种维护类型属于救火式维护

A. 预防性维护 B. 适应性维护 C. 改正性维护 D. 完善性维护

标签：C (改正性维护)。

解析：**"救火式" (firefighting) 维护指系统出现紧急故障后被动响应、临时抢修的维护活动**。教材 11.1 节定义："为了识别和纠正软件错误、改正软件性能上的缺陷而进行诊断和改正错误的过程称为**改正性维护**"，其特征是被动、紧急、诊断—改正，与"救火式"语义对应。预防性是主动的；适应性、完善性是计划内的改造，不属于救火式。

【2013-2014 期末 判断】软件的项目管理包括软件项目的启动、执行、监控和结束。

标签：× (错)。

解析：教材 12.3 节明确**项目管理有 5 大过程组：启动、计划、执行、监控、收尾**。题目遗漏了"**计划过程组**"，而计划过程组是 PM 最核心的活动之一（含制订项目管理计划、范围定义、WBS、进度计划、成本预算、风险计划等）。

【2015-2016 期中 判断】缺乏软件项目管理的经验，是产生软件危机的原因之一。

标签：√ (对)。

解析：教材 1.2.2 节"软件危机"和 1.3.1 节均指出，导致软件项目失败的主要原因并不是采纳的技术和工具，而是**不适当的管理方法**。"软件危机的解决途径"也指出"软件开发和维护所使用的方法不合理"是危机原因之一，其中即包含项目管理经验的缺乏。

【2021-2022 期末 简答】如果在软件维护周期内发现系统有设计问题，那么请问如何解决这个设计问题？在部署新代码之前有哪些软件测试活动？并请给出系统新功能部署的可能方案。

1) 如何解决设计问题（按 IEEE 1219 维护流程）：

1. 维护申请 (MR) ——发现设计问题后，由用户或开发人员提交维护申请报告，明确说明问题的现象、复现条件、影响范围；
2. 分类与鉴别——维护机构确认维护类别（理论上属**改正性维护**——设计缺陷修复；若同时优化结构则含完善性成分），分配编号入库；
3. 分析——先做可行性分析（影响范围、解决方案、工作量、风险），再做详细分析（更改需求说明、鉴别须更改的要素、提出测试策略）；评估对其他模块的副作用（11.3.2 三种副作用）；
4. 设计——更改设计基线，更新需求/设计文档、修改测试计划；如设计问题严重，参考 ch06 设计原则（高内聚低耦合、模块独立性）重构；
5. 实现——制订程序修改计划，编码、单元测试、集成；置于软件配置管理 (SCM) 控制之下；结束时做风险分析；
6. 测试——按下方"部署前测试活动"执行；
7. 验收测试——客户/用户/第三方共同进行，建立软件新版本；
8. 交付——安装、培训、备份、通知用户。

2) 部署新代码前的测试活动：

1. 静态确认 (11.3.3) ——多人 review；
2. 单元测试——白盒为主，针对修改后的模块；
3. 集成测试——验证模块间接口、组合功能；
4. 回归测试——确保修改未引入新错误、原有功能不受影响；
5. 确认测试——验证是否满足需求规格说明；
6. 系统测试——在真实运行环境下做功能、性能、安全、可靠性等测试；
7. 验收测试——用户/客户/第三方共同测试，确认可交付。

3) 新功能部署的可能方案（运维工程实践）：

1. 全量发布 (Big-bang)：停机维护，新版本替换旧版本；
2. 蓝绿部署 (Blue-Green)：两套环境切换，瞬时回滚；
3. 灰度/金丝雀发布 (Canary)：按比例逐步切换流量 (1%→10%→50%→100%)；
4. 滚动部署 (Rolling)：分批替换服务器实例；
5. A/B 测试：按用户特征分组并行运行；
6. 特性开关 (Feature Toggle)：新代码已部署但功能通过开关控制；
7. 微服务独立部署：作为独立服务发布，与主系统解耦。

无论哪种方案，均须配套**回滚机制与监控告警**，并保留旧版本以便快速回退。

第十一章 软件维护（速查）

- 定义：交付后为改正错误或满足新需求而修改软件的一切活动
- 占比：维护占生存期工作量 **70%+**；大型软件维护成本 $\approx$ 开发的 **4 倍**
- 完善性占比最大：约 **50%**
- 四类维护：
 - 改正性 $\rightarrow$ 改 bug / 救火式
 - 适应性 $\rightarrow$ 适应**环境变化**（OS/DB/硬件）
 - 完善性 $\rightarrow$ 新增功能 / **性能优化**（最大）
 - 预防性 $\rightarrow$ 主动**重构**提可维护性
- 口诀：错 $\rightarrow$ 改、境 $\rightarrow$ 适、新 $\rightarrow$ 完、防 $\rightarrow$ 预
- 影响因素：系统大小、程序设计语言、系统年龄、文档完备
- 典型问题：定位困难、理解困难、文档不足、原开发人员不在、可维护性差、修改易引入新异常
- 维护机构 **7 角色**：用户、维护管理员、系统分析员、修改负责人、测试人员、工程人员、配置管理员
- MR：用户填；SCR：内部给
- IEEE 1219 流程：申请 $\rightarrow$ 分类 $\rightarrow$ 分析 $\rightarrow$ 设计 $\rightarrow$ 实现 $\rightarrow$ 系统测试 $\rightarrow$ 验收测试 $\rightarrow$ 交付
- 维护期开发三步：理解程序 $\rightarrow$ 修改程序 $\rightarrow$ 重新验证
- 三种副作用：代码、数据、文档
- 逆向工程：分析+抽取设计（不改源码）
- 再工程：逆向+重构+正向，重新构造为更高质系统
- 维护 $\neq$ 改 Bug（仅改正性才是）
- 维护周期含多次开发活动（ $\checkmark$ ）

第十二章 软件项目管理（速查）

- 项目：时限性、唯一性、有限资源
- 软件项目特征：无形、无标准过程、一次性
- 四制约：**范围/成本/进度/客户满意度**
- 项目管理 $\neq$ 软件工程（PM 是 SE 子集）
- 九大知识域：综、范、时、成、质、人、沟、风、采
 - 综合：章程/计划/执行/监控/变更/收尾
 - 范围：规划/定义/**WBS**/确认/控制
 - 时间：定义/排序/资源估算/历时估算/进度/控制
 - 成本：估算/预算/控制
 - 质量：规划/保证/控制
 - 人力：规划/组建/建设/管理
 - 沟通：规划/发布/绩效/干系人
 - 风险：规划/识别/定性/定量/应对/监控
 - 采购：规划/招标/选择/合同/收尾
- 五过程组：启动/计划/执行/监控/收尾
- 计划过程组：内容**最多**
- 收尾：仅综合（项目收尾）+ 采购（合同收尾）
- 度量实体：产品/过程/资源
- 属性分类：内部 vs 外部；直接 vs 间接
- LOC：生产率=KLOC/人月；质量=错误/KLOC
- FP 公式： **$FP = \text{总计数} \times (0.65 + 0.01 \times \sum Fi)$**
- FP 五数据域：EI/EO/EQ/ILF/EIF
- 估算：分解/经验/COCOMO
- 期望工期： **$T = (a + 4m + b) / 6$**
- $\sigma = (b - a) / 6$
- 关键路径：PERT/CPM、甘特图
- 组织原则：尽早落实责任、减少接口、责权均衡
- 三模式：按课题/按职能/矩阵
- PM 80%+ 时间用于沟通
- 增加人力总能赶上进度 $\rightarrow \times$ （Brooks）
- CMM 是过程能力框架，非仅 PM 标准
- 缺 PM 经验 = 软件危机原因之一（ $\checkmark$ ）